VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
PRATEEK CHATURVEDI	24BCE10168
Slot	**Date of Submission**
D11	26/03/2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you

understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: Linux Kernel

3. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Do not just copy the official description. Research the original context. What was frustrating or broken or inaccessible before this software existed? Why did the creator decide to build it and share it rather than keep it proprietary?

For example: Linus Torvalds was a student who wanted a free Unix-like system and could not afford one. Richard Stallman watched a printer malfunction and could not fix it because the source code was locked. The VLC team wanted to stream video over a university network but no tool existed that was free and open enough to adapt.

What is your software's origin story? Tell it properly.

A2. The license — what it actually says

Download or find the actual license text for your software. Read it. Then answer these questions in your report:

- What are the four freedoms of free software? Does your software's license grant all four?
- Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?
- What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?
- Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

Think, don't copy	Answers copied from Wikipedia or AI tools will score zero. We are looking for your understanding and your words.

A3. The ethics of open source

Open source is not just a licensing model — it is a statement about how knowledge and tools should be shared. Reflect on the following:

- Should all software be open source? Make an argument for and against.
- Is it ethical for a company to build a profitable product entirely on top of free community labor? (Think about how companies like Red Hat, Google, and Meta use Linux.)
- What does 'standing on the shoulders of giants' mean in the context of software? Does open source enable or undermine original innovation?

There are no correct answers here. We are grading the depth of your thinking, not your conclusion.

Part B — Linux Footprint (Unit 2)

Research and document how your chosen software lives on a Linux system. Where does it install? What user does it run as? How is it managed? This section requires you to actually install the software on Linux — either on your machine, a virtual machine, or a lab system.

- Installation method — is it available via RPM, apt, or must it be compiled from source?
- Key directories — where are its binaries, config files, and logs stored? (e.g., /usr/bin, /etc, /var/log)
- User and group — what Linux user and group does it run under? Why does this matter for security?
- Service management — can it be started, stopped, or restarted as a service? Show the commands.
- Update model — how does the open-source community push security patches to your machine?

Part C — The FOSS Ecosystem (Units 3 & 4)

No open-source project exists in isolation. Map the ecosystem around your chosen software.

- What other open-source tools does it depend on? (Libraries, compilers, runtimes)
- What has it enabled or inspired? (Forks, spin-offs, tools built on top of it)
- Where does it fit in the LAMP stack, or how does it relate to the web?
- Is there a community — mailing lists, forums, conferences, IRC channels? Who governs it?

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source	Proprietary Alternative
Cost		
Security (who can audit the code?)		
Support and reliability		

Freedom to modify		
Community vs corporate control		
Your overall verdict		

Fill in the table as part of your report, then write a two-paragraph conclusion: given everything you have learned, would you recommend your chosen software for a real-world deployment? Would you contribute to it?

4. Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software

# --- Variables ---
STUDENT_NAME=""          # Fill in your name
SOFTWARE_CHOICE=""       # Fill in your chosen software

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)

# --- Display --- echo
"================================" echo "
Open Source Audit — $STUDENT_NAME" echo
"================================" echo
"Kernel  : $KERNEL" echo "User    :
$USER_NAME" echo "Uptime  : $UPTIME"
# TODO: Add distro name, date, and license message
```

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE=""   # e.g. httpd, mysql, vlc, firefox

# Check if package is installed if
rpm -q $PACKAGE &>/dev/null; then
echo "$PACKAGE is installed."
    rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
    echo "$PACKAGE is NOT installed." fi

# TODO: Add a case statement that prints a one-line # philosophy note
about the package based on its name case $PACKAGE in     httpd)  echo
"Apache: the web server that built the open internet" ;;     mysql)  echo
"MySQL: open source at the heart of millions of apps" ;;
    # TODO: Add your software and 3 others esac
```

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Starter structure

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)         echo
"$DIR => Permissions: $PERMS | Size: $SIZE"     else
```

```
        echo "$DIR does not exist on this system"
fi done

# TODO: Add a section that checks if your software's
# config directory exists and prints its permissions
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages

LOGFILE=$1
KEYWORD=${2:-"error"}    # Default keyword is 'error'
COUNT=0

if [ ! -f "$LOGFILE" ]; then     echo
"Error: File $LOGFILE not found."     exit
1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))     fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# TODO: Add a do-while style retry if the file is empty,
# and print the last 5 matching lines using tail + grep
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL read
-p "2. In one word, what does 'freedom' mean to you? " FREEDOM read
-p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# TODO: Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Software chosen (Linux Kernel), Linux VM setup completed, kernel verified using uname -r*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft of origin story (A1) explaining problem and creation of Linux Kernel*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft of license analysis (GPL v2) and ethical discussion (A2 + A3)*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Part B completed with commands, directory analysis, and screenshots from VM*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Script 1 (System Identity) and Script 2 (Package Inspector) completed and tested*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Script 3 (Disk Auditor) and Script 4 (Log Analyzer) completed with outputs*

7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Part C completed with dependencies, community, and ecosystem analysis*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Part D completed and Script 5 (Manifesto Generator) implemented*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Full report compiled (12–16 pages) with code, screenshots, and explanations*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission with GitHub repo, README.md, and PDF uploaded*

6. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original — in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story — depth, original research, storytelling	**12**	6	*Part A1*
2	License analysis — accuracy and understanding of freedoms	**12**	6	*Part A2*

3	Ethical reflection — quality of argument and original thinking	**10**	5	*Part A3*
4	Linux footprint — technical accuracy and screenshots	**10**	5	*Part B*
5	Ecosystem and LAMP connection	**8**	4	*Part C*
6	Open source vs proprietary comparison	**8**	4	*Part D*

Shell Scripts (40 marks — 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness — does it run without errors?	**4**	*Script executes on Linux and produces expected output*
Concepts — does it use the right shell constructs?	**2**	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	**2**	*Every non-obvious line has a comment; script purpose is clear*

Academic integrity	All written sections must be in your own words. Code that is copy-pasted from the internet without understanding will be zero-marked. Running a script and explaining what it does in a conversation is considered a passing demonstration of understanding.

7. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar':

catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

8. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?
GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission — partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.

Software Chosen: **Linux Kernel**

PART A — ORIGIN AND PHILOSOPHY (Units 1 & 2)

A1 The Problem the Linux Kernel Was Created to Solve

In the early 1990s, access to powerful operating systems was limited and expensive. Systems such as UNIX were primarily available to large organizations, making them inaccessible to students and independent developers. This created a significant barrier to learning and innovation.

Linus Torvalds, a computer science student, was using MINIX for educational purposes. Although useful, MINIX had limitations and restricted deeper modification. This lack of flexibility motivated him to develop a system that was free, modifiable, and suitable for personal computers.

Instead of keeping his work private, Linus released the Linux Kernel publicly in 1991 and invited contributions from developers worldwide. This decision transformed a small personal project into a global collaborative effort. The Linux Kernel solved not only a technical problem but also introduced a new philosophy based on openness, collaboration, and shared knowledge.

- At that time, Linus Torvalds was using MINIX for learning purposes. Although it was useful, it had significant limitations that restricted deeper system-level modifications. This lack of flexibility was a major drawback for someone interested in understanding and improving operating systems.
- He aimed to build a system that was free, fully modifiable, and capable of running on personal computers. Instead of waiting for better alternatives, he began developing his own kernel. Initially a small personal project, it gained attention when he shared it publicly in 1991 and invited contributions from developers worldwide.
- This marked a turning point, as the Linux Kernel evolved through global collaboration rather than centralized control. It not only solved a technical problem but also established a strong foundation for open-source development based on transparency and shared innovation.

Today, the Linux Kernel is the engine behind :

- Almost every machine online
- Android smartphones
- Supercomputers
- Embedded systems

A2 The License — What It Actually Says

The Linux Kernel is governed by the GNU General Public License version 2 (GPL v2). This specific license is a landmark in open-source history because it guarantees software is "free" in terms of *liberty* (freedom), not just price (cost).

The Four Freedoms of Free Software

The whole philosophy of the GPL is built around these four crucial freedoms:

1. The freedom to run the program for any purpose
2. The freedom to study how the program works
3. The freedom to modify the program
4. The freedom to distribute copies, including modified versions

The Linux Kernel totally honors all four of these freedoms. It means anyone can grab the source code, change it up, and pass it on to others.

Can a Company Modify and Sell Linux Without Sharing Changes?

Companies are totally free to use Linux-based systems and even sell them for a profit. However, there's one big condition: if a company tweaks the Linux Kernel and then distributes that modified version, they are required to do two things:

- Release the modified source code
- Use the same GPL license

This requirement is called *copyleft*. It's a clever mechanism that makes sure all new improvements stay open source, continually benefiting the whole community. You see this in action with companies like Red Hat, which make money selling Linux products but still follow the GPL rules by publishing their source code.

Difference Between GPL and MIT License

Feature	**GPL License**	**MIT License**
Source Code Sharing	Mandatory	Optional
Commercial Use	Allowed	Allowed
Modification Rules	Must remain open	Can become closed
Restriction Level	Strong	Minimal

Simply put, the GPL is the tougher one; it actively safeguards user freedom by demanding openness. The MIT license is far more laid-back, letting companies integrate the code into closed, proprietary systems. Personally, if I were launching my own project, I'd go with the GPL—it's the best way to ensure my work remains open and keeps helping others.

Free as in “Free Beer” vs Free as in “Freedom”

You need to understand a crucial distinction in the world of open source:

- Free as in free beer → No cost
- Free as in freedom → Freedom to use, modify, and share

Linux embodies *freedom*, not just "free of charge." So even when businesses sell systems powered by Linux, the core code is always accessible and anyone can modify it.

A3 The Ethics of Open Source

Open source isn't just about lines of code—it’s a mindset about how we share knowledge and how technology progresses over time.

Should All Software Be Open Source?

You hear strong points from both camps. For instance, people who favor it say:

- Open source promotes transparency and trust
- It allows developers to learn from existing code
- It encourages collaboration and innovation
- Security improves because anyone can inspect the code

On the flip side, arguments against include:

- Companies need revenue to sustain development
- Some software (like banking or defense systems) requires strict control
- Not all developers want to share their work freely

My take is that we need a balanced perspective. Key infrastructure, the stuff everything else relies on, should absolutely be open, but there’s room for specialized applications to remain proprietary.

Is It Ethical for Companies to Profit from Open Source?

This topic gets complicated quickly. It’s true that huge corporations like Google, Red Hat, and Meta use open-source projects like Linux to create highly profitable products. It can definitely look like they are just leveraging the community's hard work. But the reality is that many of these companies also put a lot back in:

- They improve the code
- They fix bugs
- They support the community

This system generally creates a healthy loop of giving and receiving. The real ethical issue pops up when a company uses open-source software and doesn't bother to contribute *anything* in return. In those specific cases, it starts to look less like collaboration and more like exploitation.

“Standing on the Shoulders of Giants”

When someone talks about “Standing on the Shoulders of Giants,” they mean that new ideas are fundamentally built upon the knowledge and work that came before. In the context of software development, this means:

- Developers do not start from scratch
- They use existing tools, libraries, and systems
- They improve and extend them

The Linux Kernel itself perfectly illustrates this concept—it took inspiration from UNIX but was improved and elevated by global community effort. Open source doesn't slow innovation; it actually speeds it up dramatically. By removing the need to reinvent basic systems, developers are free to concentrate their energy on truly new and cutting-edge technologies.

PART B — TECHNICAL DEPLOYMENT

B1 Installation Method of Linux Kernel

Not like normal programs, the Linux Kernel forms the heart of the system. So every Linux version—say Ubuntu, Kali, or Fedora—comes with it built right in. Frequently, updates to the core part of an operating system happen through its built-in software tool. Sometimes installation takes place alongside regular programs, handled just like any other update.

- For Debian or Ubuntu setups, use apt instead
- On Fedora → dnf
- For Arch, use pacman

Use this command when you want to see which kernel version is active right now:

```
None
uname -r
```

.

Check what kernel packages are currently on the system:

```
None
dpkg --list | grep linux-image
```

(for Debian/Ubuntu) or

```
None
rpm -qa | grep kernel
```

(for Fedora)

B2 Key Directories of the Linux Kernel

The Linux Kernel interacts with several important system directories that are essential for system operation. These directories store kernel files, modules, system data, and device interfaces. Understanding these directories The Linux Kernel interacts with several important system directories that are essential for system operation. These directories store kernel files, modules, system data, and device interfaces. Understanding these directories helps in analyzing how the kernel manages hardware and processes.. These paths matter because they shape how operations unfold behind the scenes. Each one serves a distinct role that supports core functions. Through them, communication flows between hardware and software layers. Their structure stays consistent across most installations. Without access to these locations, basic tasks would fail silently.

1. /boot — Kernel Files

This directory contains:

- Kernel image (vmlinuz)
- Initial RAM disk (initrd)
- Bootloader files

Run: ls /boot

2. /lib/modules — Kernel Modules Directory

The /lib/modules directory contains loadable kernel modules such as device drivers. Each folder corresponds to a specific kernel version and allows dynamic extension of kernel functionality.
Run : ls /lib/modules

3. /proc — Kernel Data via Virtual File System

The /proc directory is a virtual filesystem that provides runtime information about system processes and kernel status. It does not store actual files on disk but generates data dynamically when accessed.
Run : cat /proc/version

4. /sys — System and Kernel Interface

The /sys directory provides an interface to access kernel and device information in a structured format. To core components alongside physical devices.
Run: ls /sys

5. /dev — Device Files

The /dev directory contains device files that act as interfaces between hardware components and the operating system. These files allow the kernel to communicate with physical devices.
Run : ls /dev | head

6. /var/log — Logs

Here's where logs tied to the kernel land. Sent by the system when core operations leave traces behind.
Run: dmesg | tail

B3 User and Group (Security Aspect)

Most programs act like everyday users on a computer, but the Linux Kernel is different. Running deep within the machine, it takes control through privileged access, allowing full command over operations. Instead of sitting at the user level, it works behind everything as an unseen yet essential layer.

- A single person does not own it, unlike user accounts such as www-data or mysql.
- Running deep, it takes charge of hardware at the most fundamental level

Why this matters:

- Provides full control over memory, CPU, and devices
- Ensures system-level operations are handled securely
- Stopping outsiders from touching the machinery without permission

To check current user : whoami
Check whether you have root access : id

B4 Service Management

Born from code, not something you launch or shut down as one might with regular programs. Instead:

- It is loaded during system boot
- Under control of GRUB, the startup manager

Still, handling tasks tied to the kernel is possible through certain steps:

- **View loaded kernel modules:** lsmod
- **Start by bringing in a module:** sudo modprobe <module_name>
- **Remove a module:** sudo rmmod <module_name>
- **Check system services (related to kernel indirectly):** systemctl list-units --type=service

B5 Update Model (How Kernel Updates Work)

Every day, folks around the world tweak the Linux Kernel bit by bit. Changes flow in from different corners of the planet without pause.

How updates are delivered:

- Using different versions of Linux—like Ubuntu or Kali—to get things done
- Using package managers like apt, dnf, etc.

Example:

```
None
sudo apt update
sudo apt upgrade
```

These commands:

- Fetch latest kernel patches
- Install security updates

Here's what makes it strong:

- Bugs are fixed quickly
- Security vulnerabilities are patched globally
- Fresh changes show up clearly, anyone can check them

Check available kernel updates:

```
None
apt list --upgradable
```

PART C — ECOSYSTEM AND INFLUENCE

C1 Dependencies of the Linux Kernel

A single component rarely works alone. Even if it is central, everything around it breathes life through helpers that arrive before, beside, or after it. Development stumbles without companions, compilation waits for outside hands, and operation leans on unseen parts nearby.

Development Dependencies:

To build or modify the Linux Kernel, developers rely on:

- GCC (GNU Compiler Collection) → Used to compile kernel source code
- Make (Build System) → Automates compilation process
- Binutils → Handles binary operations

Built into most Linux systems, glibc handles core functions that software needs to run. Open source lies at the heart of these tools, revealing how tangled their connections really are.

Runtime Dependencies:

During execution, it's the kernel that communicates with:

- Hardware drivers (disk, network, USB)
- System libraries
- Bootloader (GRUB)

Running deep inside, the kernel links physical parts to programs, enabling everything your machine does. Without it, nothing works together at all.

C2 What the Linux Kernel Has Enabled and Inspired

The heart of countless systems beats inside the Linux Kernel. More than just code, it shapes digital worlds. It doesn't simply run machines; through quiet power beneath unseen layers, it brings life into tools, servers, and everyday devices.

1. Linux Distributions

A core piece made full operating systems possible, such as:

- Ubuntu
- Kali Linux
- Fedora
- Debian

Out of the box, these setups pair the kernel alongside everyday utilities, shaping something functional. What emerges is a working environment where core code meets practical programs, built to run tasks without extra steps.

2. Android Operating System

Billions of phones run Android, which sits atop a tweaked version of the Linux core. That quiet change helped free code go worldwide.

3. Cloud and Servers

Most web servers and cloud platforms rely on Linux:

- AWS
- Google Cloud
- Microsoft Azure

Behind much of today's online world sits Linux, holding things together without drawing attention.

4. Supercomputers

Faster than nearly every machine out there, most top-tier supercomputers choose operating systems built on Linux, mainly because it bends without breaking while moving at high speeds.

5. Embedded Systems and IoT

Linux is used in :

- Smart TVs
- Routers
- Cars
- Industrial systems

From farming to finance, it shifts shape without missing a beat.

C3 Relation to the LAMP Stack and the Web

The Linux Kernel is the base layer that keeps everything running behind the scenes in the LAMP stack, which stands for:

- L stands for Linux - that includes the kernel along with the operating system
- A → Apache (Web Server)

- M → MySQL (Database)
- P → PHP (Programming Language)

In this stack, Linux provides the operating system, taking control of machine parts to run tasks smoothly. While handling operations, it directs how components work together, ensuring that programs run efficiently. Holding things steady helps keep web apps safe, as a firm setup blocks trouble before it starts. Running the modern web would be impossible without Linux holding things together behind the scenes.

C4 Community and Governance

A worldwide network backs the Linux Kernel, giving it strength. People everywhere contribute in their own way. This shared effort shapes how the system grows. Support comes from many countries, not just a few. The reach extends far beyond any single team.

1. Developer Community

Thousands of developers contribute worldwide:

- Folks chip in. Firms toss funds too
- Code is reviewed before being accepted

2. Governance Model

The Linux Kernel Uses a Tiered System for Contributions:

- Linus Torvalds → Maintains overall control
- People who keep things running take care of particular parts of the system
- People help by sending code changes

Openness stays protected because quality checks happen at the same time.

3. Communication Channels

The Linux Community Uses:

- Mailing lists (Linux Kernel Mailing List - LKML)
- Git repositories
- Forums and documentation

Mail threads stick around longer than chats, so Linux folks keep using them to talk code. That way, anyone can follow what changed and why - no messages vanish overnight. Conversations unfold slowly, sure, yet every decision leaves a clear trail behind. Instead of quick replies piling up, each point gets space to breathe. History stays put, accessible, never locked inside an app nobody remembers.

4. Conferences and Events

Meetups help push Linux forward. Alongside these gatherings, conferences play a role too. Hackathons also add momentum throughout the year. Workshops pop up now and then to bring people together. Online sprints contribute just as much behind the scenes:

- Linux Foundation events
- Open Source Summit
- Developer conferences

Working together sparks new ideas. Team efforts open doors to fresh solutions.

C5 Role of Companies in the Ecosystem

Big firms pour significant time and resources into Linux. Despite it being free to tweak, corporations back it heavily, and tech giants contribute code regularly. Enterprise support remains strong because while the code is open to all, businesses still find immense value in investing in its stability.

- Google → Uses Linux in Android
- Red Hat → Enterprise Linux solutions
- Hardware gains strength through Intel's fine tuning
- IBM → Cloud and enterprise contributions

This is what happens when two parts come together like that : **Community + corporations work together**

PART D — OPEN SOURCE VS PROPRIETARY

Comparison Table (Linux Kernel vs Proprietary OS)

Dimension	**Open Source (Linux Kernel)**	**Proprietary Alternative (Windows/macOS)**
Cost	Free to use, distribute, and modify. No licensing fees required.	Requires paid licenses (Windows) or restricted to specific hardware (macOS).
Security (who can audit the code?)	Source code is publicly available, so anyone can audit, identify vulnerabilities, and contribute fixes.	Source code is closed; only the company can inspect and fix security issues.
Support and reliability	Supported by a global community and enterprise vendors like Red Hat. Known for high stability, especially in servers.	Official support provided by companies like Microsoft and Apple. Reliable but dependent on vendor updates.
Freedom to modify	Full freedom to modify, customize, and redistribute under GPL license.	No freedom to modify core system; users are restricted to vendor-provided features.

Dimension	Open Source (Linux Kernel)	Proprietary Alternative (Windows/macOS)
Community vs corporate control	Governed by a global community with contributions from individuals and companies. Decisions are transparent.	Fully controlled by a single corporation with centralized decision-making.
Your Overall Verdict	More suitable for servers, developers, and enterprise environments due to flexibility, security, and cost-effectiveness.	More suitable for general users who prefer ease of use, polished interface, and official support.

Two-paragraph conclusion : given everything you have learned, would you recommend your chosen software for a real-world deployment? Would you contribute to it?

When you look closely, Linux stands out because it works well in actual use - think servers, cloud setups, security tools, and coding workspaces. Its source being open means anyone can check how it runs, spot issues, then help patch them up; this teamwork across continents sharpens its strength slowly but surely. People building software often find value in that openness, since improvements come from many different directions at once. Not needing to pay fees to run it gives companies an edge, especially those watching budgets while scaling fast. Over months or years, small changes add up into something steady and tough to break. Being able to tweak the core parts of Linux to fit exact needs makes it stronger than closed systems that rarely allow changes. Still, working with Linux can demand deeper know-how, particularly for those just starting out. On the flip side, operating systems such as Windows or macOS tend to offer smoother experiences through simpler designs and help desks ready to assist. Even so, when speed, safety, and room to grow matter most, picking the Linux Kernel often turns out smarter in real-world settings.

Looking at it one way, the Linux Kernel stands out when putting software into actual use, especially places that need precision, speed, leaving room for growth. Running on everything from giant computers down to tiny gadgets shows how

tough and flexible it really is. Because the source code opens up, people see exactly what happens inside while also getting chances to grow skills by working with real examples instead of guessing. New ideas tend to come easier when past work supports future steps, sharing understanding freely along the way. Truth is, I see real worth in this method because it fits how tech moves forward when people work together. Down the line, jumping into the Linux world feels like a path worth taking. Sure, touching the kernel itself needs deep know-how, yet helping out can happen in different shapes - spotting flaws, sharpening guides, or lending time to connected open tools. Joining this wide network wouldn't just stretch what I can do with code, it'd tie me to an effort that reaches countless people across the planet. In the end, the Linux Kernel isn't merely lines of programming - it breathes a mindset built on sharing, teamwork, and fresh thinking.

Script 1 — System Identity Report (Unit 1 & 2)

None

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Prateek Chaturvedi | Course: Open Source Software
# --- Variables ---
STUDENT_NAME="Prateek Chaturvedi"
SOFTWARE_CHOICE="Linux Kernel"
# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2)
CURRENT_DATE=$(date '+%d-%m-%Y %H:%M:%S')
# --- Display ---
echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE_CHOICE"
echo "--------------------------------"
echo "Kernel   : $KERNEL"
```

```
echo "User      : $USER_NAME"
echo "Uptime    : $UPTIME"
echo "--------------------------------"
echo "Distro    : $DISTRO"
echo "Date      : $CURRENT_DATE"
echo "--------------------------------"
echo "License   : Linux Kernel is licensed under GPL v2"
echo "================================"
```

```
└─$ nano script1_system_identity.sh

┌──(kali㉿kali)-[~]
└─$ ./script1_system_identity.sh
================================
 Open Source Audit — Prateek Chaturvedi
================================
Software : Linux Kernel
--------------------------------
Kernel   : 6.18.12+kali-amd64
User     : kali
Uptime   : up 8 minutes
--------------------------------
Distro   : "Kali GNU/Linux Rolling"
Date     : 26-03-2026 01:02:21
--------------------------------
License  : Linux Kernel is licensed under GPL v2
================================
```

Explanation Script 1 :

This script displays key system information in a structured format. It retrieves details such as kernel version, logged-in user, system uptime, distribution name, and current date using standard Linux commands. These values are stored in variables and

displayed using formatted output. The script demonstrates basic scripting concepts such as variables, command substitution, and output formatting, while also reflecting the transparency of open-source systems. for your Linux system, designed to show key details clearly and without fuss. It embodies the open-source philosophy by putting system information right up front. It's built for function, making complex facts—like who built the kernel or how long the machine has been running—easy for anyone to grasp.
The script starts with the shebang `#!/bin/bash` to make sure it runs correctly. It then sets simple variables like `STUDENT_NAME` and `SOFTWARE_CHOICE`, which is standard practice for flexible scripting.
The real work happens when the script uses **command substitution** (`$()`) to grab dynamic system data:

- `KERNEL=$(uname -r)`: Fetches the exact kernel version.
- `USER_NAME=$(whoami)`: Identifies the active user.
- `UPTIME=$(uptime -p)`: Reports how long the machine has been running.
- `DISTRO=$(cat /etc/os-release...)`: Parses a config file to find the OS name.
- `CURRENT_DATE=$(date...)`: Captures the current date and time.

All these details are neatly displayed using `echo` and dividers, concluding with the note that the Linux Kernel uses the GPL v2 license. The point of the script is to show how simple commands can tap into deep system information, proving the power and openness of the Linux command line.

Script 2 — FOSS Package Inspector (Unit 2)

```
None
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Prateek Chaturvedi
PACKAGE="linux-image-amd64"
# Check if package is installed
if dpkg -l | grep -qw $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -l | grep $PACKAGE | awk '{print "Version:", $3}'
else
```

```
    echo "$PACKAGE is NOT installed."
fi
# Case statement for philosophy note
case $PACKAGE in
    httpd)
        echo "Apache: the web server that built the open
internet"
        ;;
    mysql)
        echo "MySQL: open source at the heart of millions of
apps"
        ;;
    vlc)
        echo "VLC: a powerful media player built by students and
community"
        ;;
    firefox)
        echo "Firefox: a browser that promotes privacy and open
web"
        ;;
    linux-image-amd64)
echo "Linux Kernel: the foundation of all Linux systems and open
computing"
;;
*)
echo "Unknown package: part of the open-source ecosystem"
;;
esac
```

```
┌──(kali㉿kali)-[~]
└─$ ./script2_package_inspector.sh
linux-image-amd64 is installed.
Version: 6.18.12-1kali1
Linux Kernel: the foundation of all Linux systems and open computing
```

Explanation of Script 2 :

This script checks whether a specific open-source package is installed on the system and displays its version information. It uses an if-else condition along with the dpkg -l command and grep to search for the package in the installed package list. If the package is found, the script extracts its version using awk and displays it. Otherwise, it prints that the package is not installed. A case statement is then used to provide a short description based on the package name. This script demonstrates conditional statements, command piping, and basic text processing in scripting.

Script 3 — Disk and Permission Auditor (Unit 2)

None

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Prateek Chaturvedi
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "----------------------"
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done
echo "----------------------"
echo "Linux Kernel Directory Check"
echo "----------------------"
# Check kernel-related directory
KERNEL_DIR="/lib/modules/$(uname -r)"
if [ -d "$KERNEL_DIR" ]; then
```

```
    PERMS=$(ls -ld $KERNEL_DIR | awk '{print $1, $3, $4}')
    echo "$KERNEL_DIR exists."
    echo "Permissions: $PERMS"
else
    echo "$KERNEL_DIR does not exist."
fi
```

```
┌──(kali㉿kali)-[~]
└─$ ./script3_disk_auditor.sh
Directory Audit Report
──────────────────────
/etc ⇒ Permissions: drwxr-xr-x root root | Size: 16M
/var/log ⇒ Permissions: drwxr-xr-x root root | Size: 1002M
/home ⇒ Permissions: drwxr-xr-x root root | Size: 6.1G
/usr/bin ⇒ Permissions: drwxr-xr-x root root | Size: 894M
/tmp ⇒ Permissions: drwxrwxrwt root root | Size: 8.0K
──────────────────────
Linux Kernel Directory Check
──────────────────────
/lib/modules exists.
Permissions: drwxr-xr-x root root
```

Explanation of Script 3 :
This script audits important system directories by displaying their permissions, ownership, and disk usage. It uses a for loop to iterate through a list of directories such as /etc, /var/log, and /home. For each directory, an if condition checks whether it exists. If it exists, the script uses ls -ld with awk to extract permissions and ownership details, and du -sh with cut to calculate the directory size. If the directory does not exist, it prints an appropriate message. Additionally, the script checks the kernel-related directory and displays its permissions. This demonstrates loops, conditional statements, and command-line data processing.

Script 4 — Log File Analyzer (Units 2 & 5)

```
None
#!/bin/bash
# Script 4: Log File Analyzer
```

```
# Author: Prateek Chaturvedi
# Usage: ./script4_log_analyzer.sh /var/log/syslog error
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi
# Check if file is empty (do-while style retry)
while [ ! -s "$LOGFILE" ]; do
    echo "File is empty. Waiting for logs..."
    sleep 2
done
# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
# Print last 5 matching lines
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

```
┌──(kali㉿kali)-[~]
└─$ ./script4_log_analyzer.sh /var/log/dpkg.log error
Keyword 'error' found 27 times in /var/log/dpkg.log
Last 5 matching lines:
2026-03-22 23:43:56 status installed libgpg-error0:i386 1.59-4
2026-03-22 23:47:54 configure libgpg-error-l10n:all 1.59-4 <none>
2026-03-22 23:47:54 status unpacked libgpg-error-l10n:all 1.59-4
2026-03-22 23:47:54 status half-configured libgpg-error-l10n:all 1.59-4
2026-03-22 23:47:54 status installed libgpg-error-l10n:all 1.59-4
```

Explanation of Script 4 :

This script analyzes a log file to count how many lines contain a specific keyword such as error or warning. It takes the log file path as a command-line argument and uses a default keyword if none is provided. The script first checks whether the file exists using an if condition. It then ensures the file is not empty using a loop. A while-read loop is used to read the file line by line, and an if condition checks each line for the keyword using grep. A counter variable keeps track of the matches. Finally, the script displays the total count and prints the last five matching lines.

Script 5 — The Open Source Manifesto Generator (Unit 5)

None

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Prateek Chaturvedi
# Alias concept (example: alias ll='ls -la') can be used to
simplify commands
echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? "
FREEDOM
read -p "3. Name one thing you would build and share freely: "
BUILD
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"
# Compose paragraph and write to file
echo "On $DATE, I reflect on the importance of open source
software. Tools like $TOOL play a vital role in my daily work,
enabling productivity and innovation. To me, freedom means
$FREEDOM, which is essential for creativity and collaboration in
technology. In the future, I would like to build $BUILD and share
```

```
it freely with the community. I believe that open source empowers
individuals and drives collective progress." > $OUTPUT
echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

```
┌──(kali㉿kali)-[~]
└─$ ./script5_manifesto.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Kali linux
2. In one word, what does 'freedom' mean to you? doing what you want to do
3. Name one thing you would build and share freely: sports complex

Manifesto saved to manifesto_kali.txt

On 26 March 2026, I reflect on the importance of open source software. Tools lik
e Kali linux play a vital role in my daily work, enabling productivity and innov
ation. To me, freedom means doing what you want to do, which is essential for cr
eativity and collaboration in technology. In the future, I would like to build s
ports complex and share it freely with the community. I believe that open source
 empowers individuals and drives collective progress.
```

Explanation of Script 5 :
This script generates a personalized open-source manifesto by taking user input. It uses the read command to ask three questions and stores the responses in variables. These inputs are then combined to form a paragraph using string concatenation. The date command is used to include the current date in the output. The script saves the generated content into a text file using output redirection and displays it using the cat command. It also includes a comment showing the use of aliases. This script demonstrates user interaction, file handling, and dynamic content generation in scripting.